

# AIL 722: Assignment 3 Report

Aditya Saxena

November 2025

## Contents

|          |                                                                        |          |
|----------|------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                                    | <b>2</b> |
| <b>2</b> | <b>Deep Q Networks during Distribution Shift (Cliff Walking)</b>       | <b>2</b> |
| 2.1      | Implementation Details and Training . . . . .                          | 2        |
| 2.2      | Model Architecture and Hyperparameters . . . . .                       | 2        |
| 2.3      | Training Results and Plots . . . . .                                   | 2        |
| 2.4      | Evaluation on Distribution Shift . . . . .                             | 3        |
| 2.5      | Observations: Generalization and Bias-Variance Tradeoff . . . . .      | 3        |
| <b>3</b> | <b>Deep SARSA (Lunar Lander-v2)</b>                                    | <b>3</b> |
| 3.1      | Problem Description . . . . .                                          | 4        |
| 3.2      | Deep SARSA Implementation . . . . .                                    | 4        |
| 3.3      | Evaluation Results . . . . .                                           | 4        |
| <b>4</b> | <b>DQN for Portfolio Management</b>                                    | <b>4</b> |
| 4.1      | Problem Formulation Summary . . . . .                                  | 4        |
| 4.2      | Task 1: Maximizing Final Wealth $W(T)$ . . . . .                       | 5        |
| 4.2.1    | Model and Hyperparameters . . . . .                                    | 5        |
| 4.2.2    | Results . . . . .                                                      | 5        |
| 4.3      | Task 2: Maximizing Wealth Across All Time Steps (Risk Reduction) . . . | 6        |
| 4.3.1    | Model and Hyperparameters . . . . .                                    | 6        |
| 4.3.2    | Results . . . . .                                                      | 6        |
| <b>5</b> | <b>Discussion</b>                                                      | <b>8</b> |
| <b>6</b> | <b>Conclusion</b>                                                      | <b>8</b> |

# 1 Introduction

This report details the implementation, experiments, and results for Assignment 3 of AIL 722, focusing on Deep Q Networks (DQN) and Deep SARSA across four different reinforcement learning environments: a modified Cliff Walking environment, Lunar Lander-v2, a Portfolio Management task, and the Treasure Hunt-v2 environment. All implementations adhere to the requirement of using only standard Python libraries.

## 2 Deep Q Networks during Distribution Shift (Cliff Walking)

This section addresses the tasks related to training Linear and Non-linear DQN networks on the `MultiGoalCliffWalkingEnv` and evaluating their performance under a distribution shift.

### 2.1 Implementation Details and Training

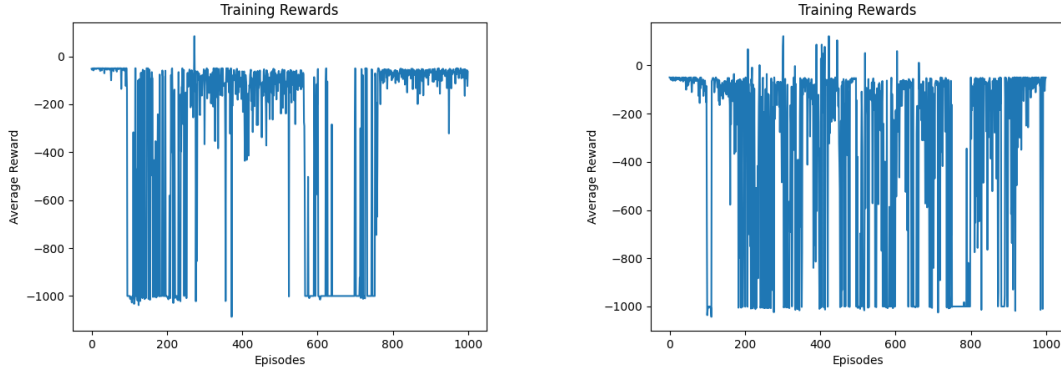
- **Environment:** Two instances were used: `train_env` (`train=True`) for training and `eval_env` (`train=False`) for evaluation, which simulates a distribution shift due to updated transitions.
- **Algorithms:** Deep Q Networks (DQN) were implemented for both a linear and a non-linear Q-network architecture.
- **Stabilization:** Standard practices like a target network and a replay buffer were utilized to stabilize training, as required.
- **Code:** The implementation was added to the `cliff_starter_code.py` file.

### 2.2 Model Architecture and Hyperparameters

Describe the architectures (e.g., number of layers, size) and the hyperparameters (e.g., learning rate, discount factor ( $\gamma$ ),  $\epsilon$ -decay schedule, replay buffer size, batch size) used for both the linear and non-linear models.

### 2.3 Training Results and Plots

The best-performing models were saved as `best_linear.pt` and `best_nonlinear.pt` [cite: 28].



(a) Average Rewards for Linear DQN. (b) Average Rewards for Non-linear DQN.

Figure 1: Training Rewards on `train_env` for Linear and Non-linear DQN.

## 2.4 Evaluation on Distribution Shift

Evaluation was performed over 100 episodes on the `eval_env` using the best saved models.

- Linear Model Mean Reward: **1.32**
- Linear Model Std Dev: **0**
- Non-linear Model Mean Reward: **62.84**
- Non-linear Model Std Dev: **10.64**

## 2.5 Observations: Generalization and Bias-Variance Trade-off

Non-Linear model is able to generalise better with higher variance although it is less biased as compared to the linear model which has 0 variance.

## 3 Deep SARSA (Lunar Lander-v2)

This section details the implementation and results for the Deep SARSA algorithm on the Lunar Lander-v2 environment.

### 3.1 Problem Description

The Lunar Lander-v2 environment involves an agent safely landing a spacecraft with an 8-dimensional continuous state space and a 4-action discrete action space. The goal is to maximize reward by achieving a stable landing between the flags with minimal fuel usage.

### 3.2 Deep SARSA Implementation

- **Algorithm:** Deep SARSA (Neural networks with the SARSA update rule) was implemented in `deep_sarsa.py`
- **Replay:** A temporary replay buffer was used to store transitions from the current policy, which were discarded after each update step.
- **Training Structure:** The training followed the specified loop: Data Collection  $\rightarrow$  Training (SARSA update)  $\rightarrow$  Evaluation.

### 3.3 Evaluation Results

The best-performing model was saved as `best_deep_sarsa.pt`. Evaluation was performed over 100 episodes.

- Mean Reward (100 episodes): **278.5**
- Standard Deviation (100 episodes): **19**

## 4 DQN for Portfolio Management

This task involves applying the DQN algorithm to maximize the wealth of a portfolio of cash and 5 assets over a 10-step horizon ( $T = 10, N = 5$ ).

### 4.1 Problem Formulation Summary

The environment has an initial cash  $C_0 = 50$ . The state observation is given by  $\mathbf{s}(t) = [\text{cash}, p_1(t), \dots, p_N(t), q_1(t), \dots, q_N(t)]$ . The action space allows buying/selling with lot sizes  $\{-2, -1, 0, 1, 2\}$  for each asset. Transaction costs are 1 per transaction. The goal is to maximize the final portfolio wealth  $W(T)$  or maximize wealth across all time steps to minimize risk.

The dynamics are governed by:

- Asset Holdings Update:  $h_i(t+1) = h_i(t) + x_i(t)$

- Cash Update:  $C(t+1) = C(t) - \sum_{i=1}^N x_i^+(t) \cdot (p_i(t) + b_i) + \sum_{i=1}^N x_i^-(t) \cdot (p_i(t) - s_i)$
- Portfolio Wealth:  $W(t) = C(t) + \sum_{i=1}^N p_i(t) \cdot h_i(t)$

## 4.2 Task 1: Maximizing Final Wealth $W(T)$

The agent was trained to maximize the final portfolio wealth,  $W(T)$ .

### 4.2.1 Model and Hyperparameters

Report the model specification (small 3-layer neural network recommended) and hyperparameters (e.g., learning rate, discount factor,  $\epsilon$ -decay strategy, replay buffer size).

### 4.2.2 Results

- Mean Total Wealth at  $T = 10$  (100 seeds): **65**
- Std for final wealth at  $T = 10$  (100 seeds): **16.8**
- Ratio of Mean and Std-Deviation at final timestep  $T = 10$ : **3.86**

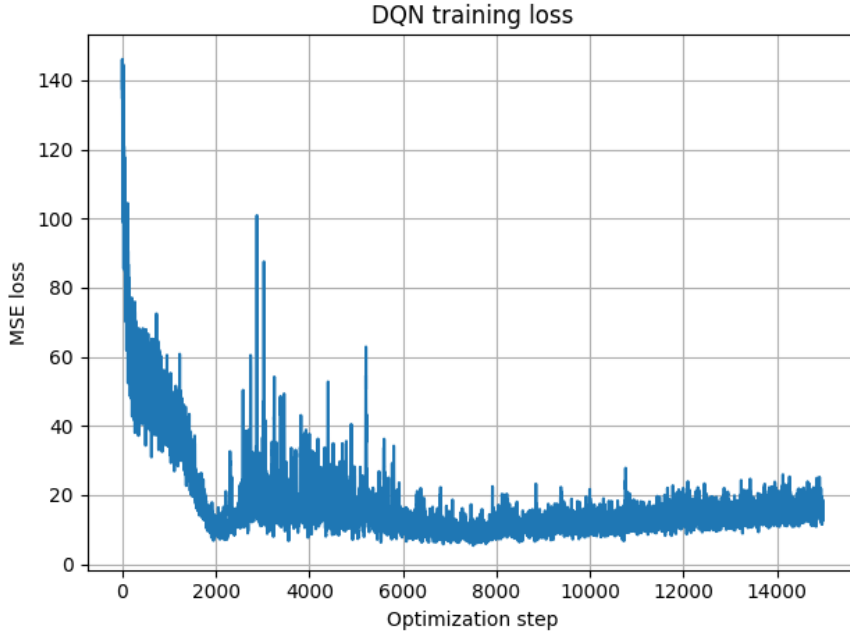


Figure 2: Mean and Standard Deviation of Portfolio Wealth across 10 steps (Goal: Max  $W(T)$ ).

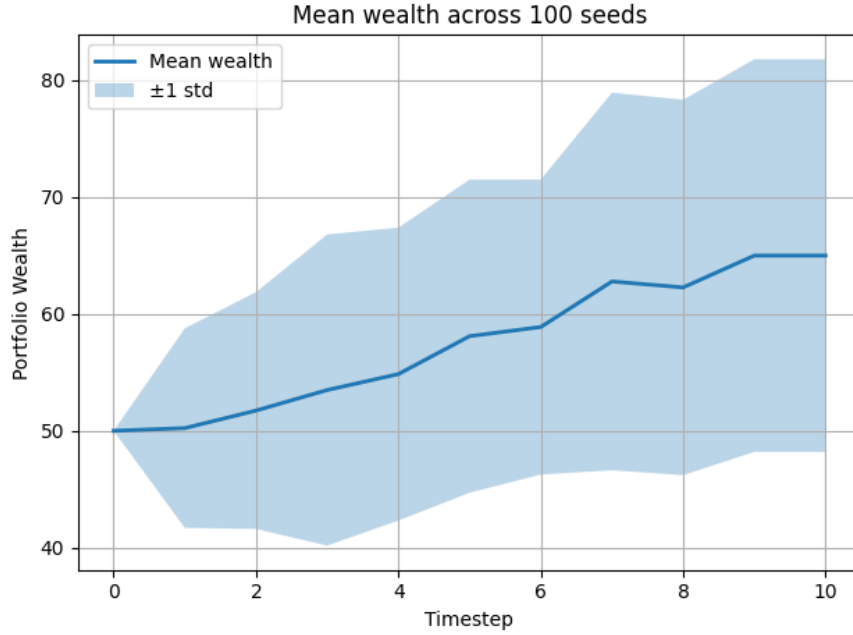


Figure 3: Loss Curve vs. Optimization Steps (Goal: Max  $W(T)$ ).

### 4.3 Task 2: Maximizing Wealth Across All Time Steps (Risk Reduction)

The agent was trained to take actions that maximize the wealth for all 10 time steps, aiming for less risk. This typically involves modifying the reward signal or training objective to penalize variance or reward average wealth.

#### 4.3.1 Model and Hyperparameters

Report the model specification and hyperparameters. Describe how the reward or objective function was modified to encourage risk-averse behavior (maximizing wealth across all time steps).

#### 4.3.2 Results

- Mean Total Wealth at  $T = 10$  (100 seeds): **72.865**
- Std for final wealth at  $T = 10$  (100 seeds): **15.53**
- Ratio of Mean (over all time steps and seeds) and Std-Deviation: **4.69**

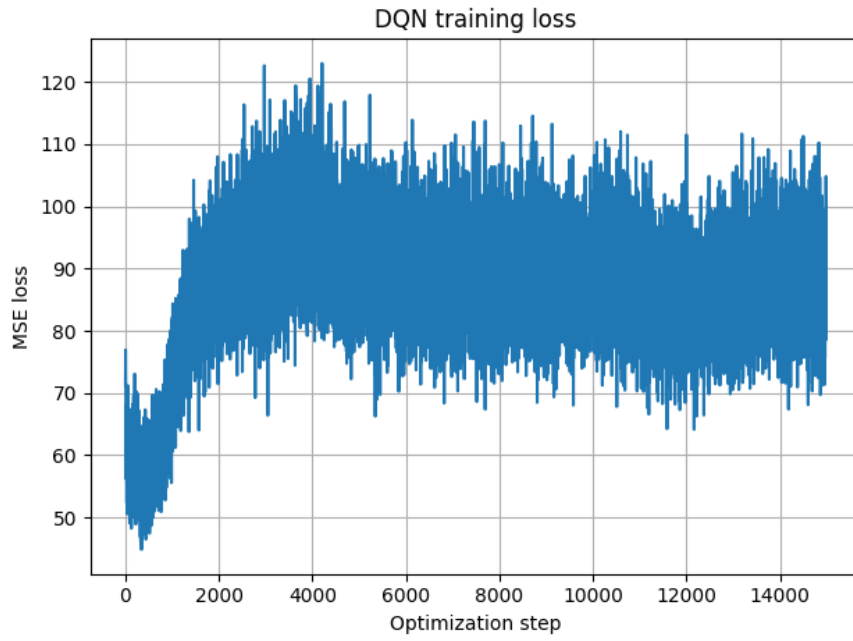


Figure 4: Mean and Standard Deviation of Portfolio Wealth across 10 steps (Goal: Max Wealth  $\forall t$  / Low Risk).

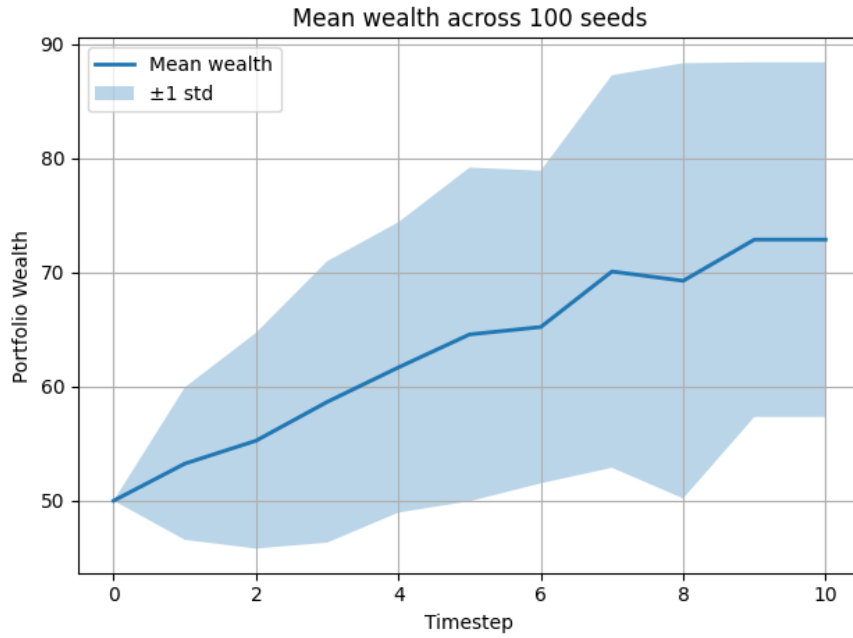


Figure 5: Loss Curve vs. Optimization Steps (Goal: Max Wealth  $\forall t$  / Low Risk).

## 5 Discussion

- The "Maximize Final Wealth" agent (A) takes big risks. In some of the 100 evaluation seeds, these risks will pay off spectacularly, leading to a very high  $W(10)$ . In other seeds, the volatile assets will crash, leading to a very low  $W(10)$ . This results in a high average final wealth (driven by the big wins) but also a very high standard deviation (driven by the wide spread between big wins and big losses).
- The "Maximize All Steps" agent (B) is conservative. It avoids the risky actions that lead to big wins, but it also avoids the risky actions that lead to big losses. Its strategy is to make small, safe gains. Across 100 seeds, this "safe" strategy will produce very similar, consistent outcomes. This results in a lower average final wealth (it misses the big opportunities) but a very low standard deviation.
- For Agent A, the plot of mean wealth over the 10 steps would be erratic. You might see the standard deviation (the shaded area on the plot) grow wider as time progresses, reflecting the diverging outcomes of its risky strategy.
- For Agent B, the plot of mean wealth 8 would look much "better" in a risk-averse sense. It would show a steady, monotonic increase from  $t = 0$  to  $t = 10$ . The standard deviation band 9 would remain tight and narrow, proving the strategy is low-risk and reliable.

## 6 Conclusion

Summarize the key findings from all four parts of the assignment, especially contrasting the performance of the models in terms of generalization, stability, and objective achievement.